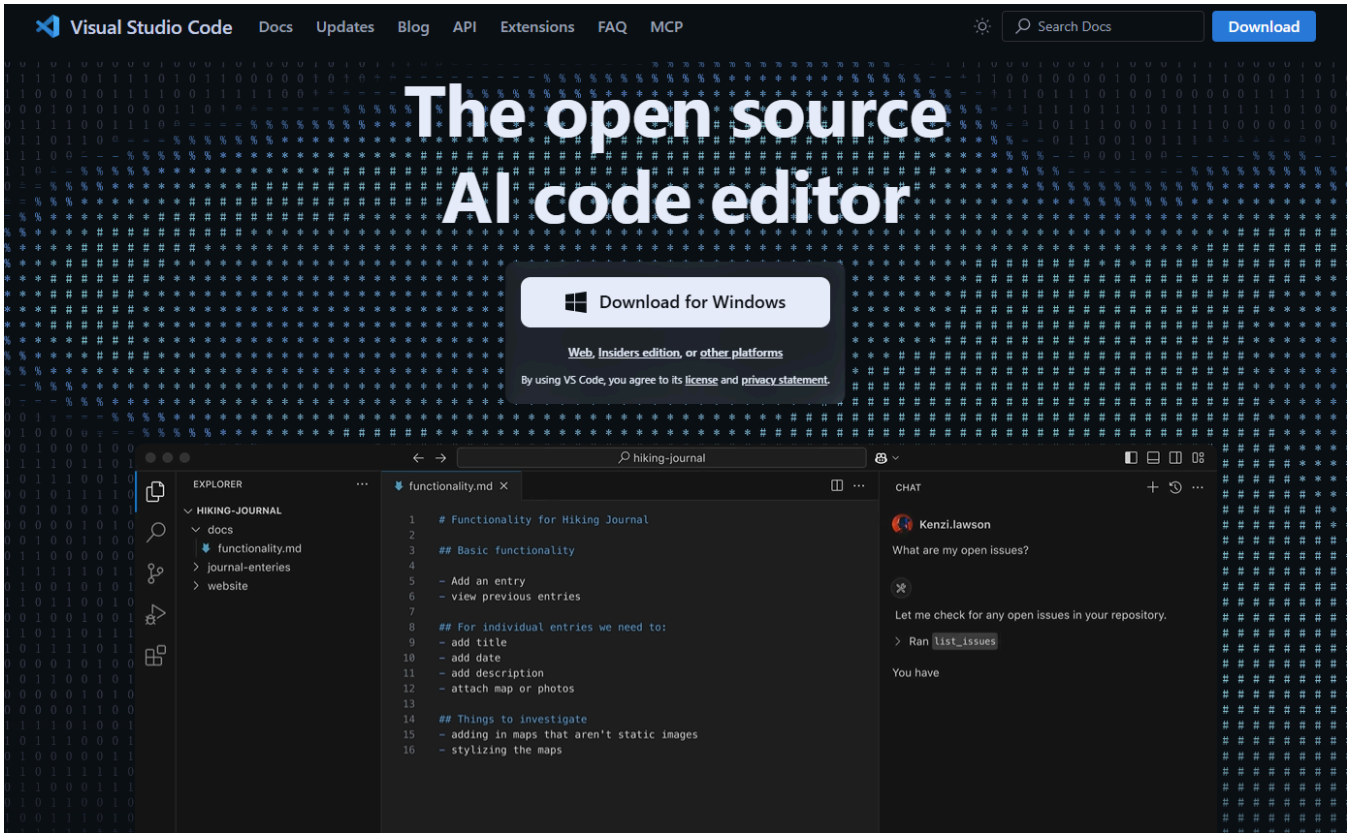
3. ROS2 Integrated Development Environment Setup

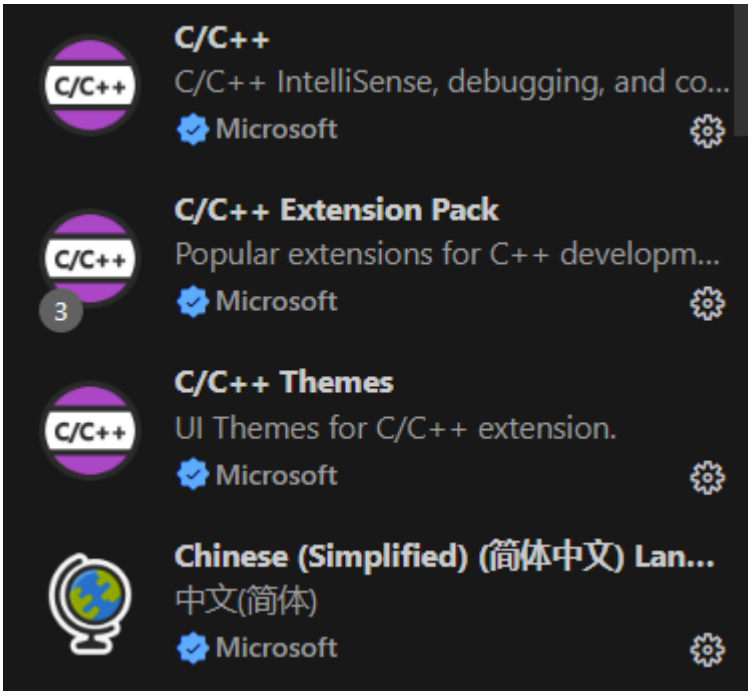
In theory, a basic ROS2 program can be written with just a text editor in ROS2, but to improve development efficiency, you can install the integrated development environment VSCode.

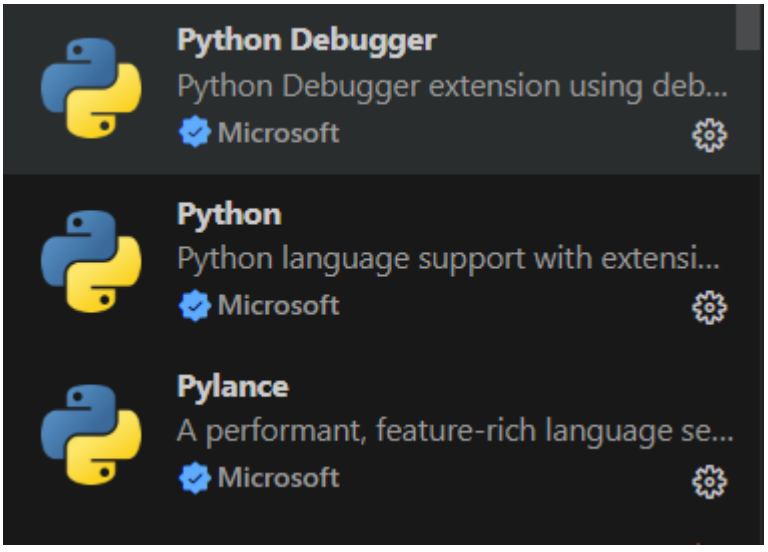
1. Using the VSCode development environment

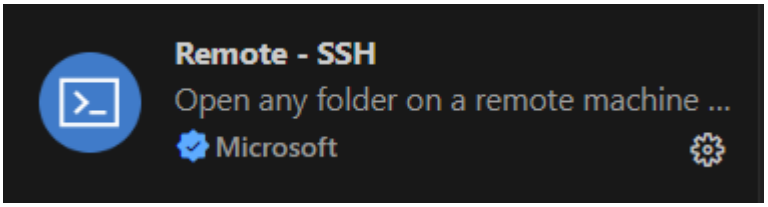
- Search for VSCode in the browser, select the installation package matching your system platform, download and install it



- Install commonly used plug-ins to help improve work efficiency

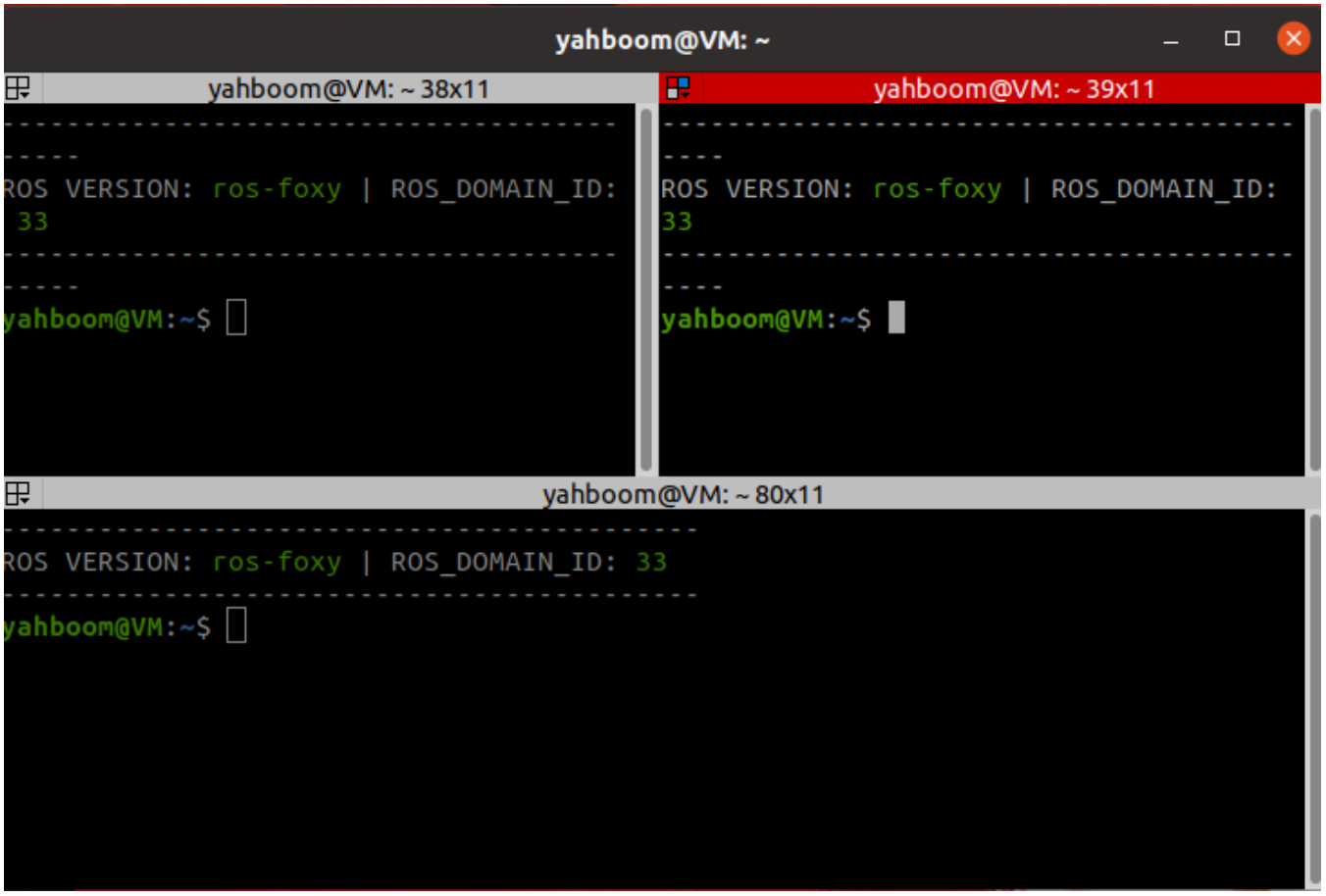






2. Using the Terminal

In ROS2, the terminal is used very frequently. A relatively easy-to-use terminal is recommended: Terminator. The result is as follows:



2.1. Installation

```
sudo apt install terminator
```

2.2. Launching

```
Shortcut `Ctrl+Alt+T` to launch
```

2.3. Common Terminator Shortcuts

Operations within the same tab:

```
Alt+Up                      // Move to the terminal above
Alt+Down                    // Move to the terminal below
Alt+Left                    // Move to the terminal on the left
Alt+Right                   // Move to the terminal on the right
Ctrl+Shift+O                // Split the terminal horizontally
Ctrl+Shift+E                // Split the terminal vertically
Ctrl+Shift+Right            // Move the splitter to the right in a vertically split
terminal
Ctrl+Shift+Left             // Move the splitter to the left in a vertically split
terminal
Ctrl+Shift+Up               // Move the splitter upward in a horizontally split terminal
Ctrl+Shift+Down             // Move the splitter downward in a horizontally split
terminal
Ctrl+Shift+S                // Hide/show the scroll bar
```

```
Ctrl+Shift+F                    // Search
Ctrl+Shift+C                    // Copy the selected content to the clipboard
Ctrl+Shift+V                    // Paste the clipboard content here
Ctrl+Shift+W                    // Close the current terminal
Ctrl+Shift+Q                    // Exit the current window, all terminals in the current
window will be closed
Ctrl+Shift+X                    // Maximize the current terminal
Ctrl+Shift+Z                    // Maximize the current terminal and enlarge the font
Ctrl+Shift+N or Ctrl+Tab        // Move to the next terminal
Ctrl+Shift+P or Ctrl+Shift+Tab  // Crtl+Shift+Tab moves to the previous terminal
```

Operations between the different tabs:

```
F11                             // Full screen switch
Ctrl+Shift+T                    // Open a new tab
Ctrl+PageDown                   // Move to the next tab
Ctrl+PageUp                     // Move to the previous tab
Ctrl+Shift+PageDown             // Swap the current tab with the tab after it
Ctrl+Shift+PageUp               // Swap the current tab with the tab before it
Ctrl+Plus (+)                   // Increase the font size
Ctrl+Minus (-)                  // Decrease the font size
Ctrl+Zero (0)                   // Restore the font to its original size
Ctrl+Shift+R                    // Reset the terminal state
Ctrl+Shift+G                    // Reset the terminal state and clear the screen
Super+g                         // Bind all terminals so that input to one is also entered
into all terminals
Super+Shift+G                   // Unbind
Super+t                         // Bind all terminals in the current tab so that content
entered into one terminal is automatically entered into the other terminals
Super+Shift+T                   // Unbind
Ctrl+Shift+I                    // Open a new window; the new window uses the same process
as the original window
Super+i                         // Open a new window; the new window uses a different
process from the original window
```

3. Using Git

3.1. Installation

In daily work, since everything is done through team collaboration and involves version management, Git is an indispensable skill. Git is a free and open source distributed version control system. To install Git under Ubuntu:

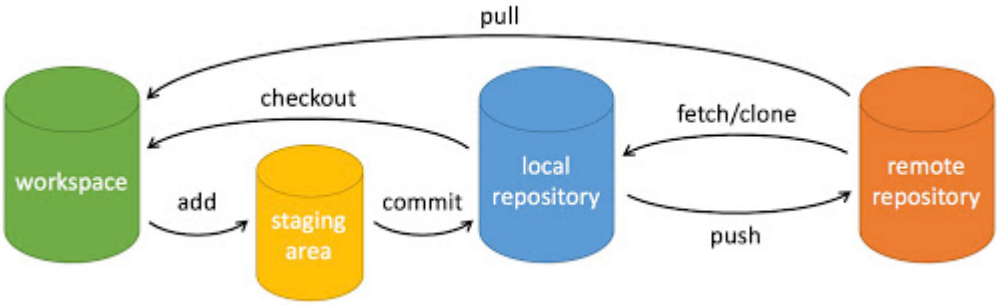
```
sudo apt install git
```

3.2. Basic Git Operations

The job of Git is to create and save snapshots of your project and compare them with later snapshots.

This chapter introduces the commands for creating and committing snapshots of your project.

The following 6 commands are commonly used in Git: **git clone**, **git push**, **git add**, **git commit**, **git checkout**, **git pull**. They will be described in detail later.



Description:

- workspace: workspace
- staging area: staging area / cache area
- local repository: repository or local repository
- remote repository: remote repository

A simple set of operating steps:

```
$ git init
$ git add .
$ git commit
```

- git init - Initialize the repository.
- git add . - Add files to the staging area.
- git commit - Add the contents of the staging area to the repository.

3.2.1. Repository Creation Commands

The following table lists the Git commands for creating a repository:

Command	Description
`git init`	Initialize the repository
`git clone`	Make a copy of a remote repository, that is, download a project.

3.2.2. Committing and Modifying

The job of Git is to create and save snapshots of your project and compare them with later snapshots.

The following table lists the commands for creating and committing snapshots of your project:

Command	Description
`git add`	Add files to the staging area
`git status`	View the current status of the repository, showing the files that have changed.
`git diff`	Compare the differences of files, that is, the difference between the staging area and the working directory.
`git commit`	Commit the staging area to the local repository.
`git reset`	Roll back a version.
`git rm`	Delete a file from the staging area and the working directory.
`git mv`	Move or rename a file in the working directory.
`git checkout`	Switch branches.
`git switch (introduced in Git 2.23)`	Switch branches more clearly.
`git restore (introduced in Git 2.23)`	Restore or undo the changes of a file.

3.2.3. Commit Log

Command	Description
`git log`	View the historical commit records
`git blame <file>`	View the historical modification records of a specified file in list form

3.2.4. Remote Operations

Command	Description
`git remote`	Remote repository operations
`git fetch`	Fetch the code repository from the remote
`git pull`	Download the remote code and merge
`git push`	Upload the remote code and merge

For more about using the git tools, you can enter `git --help` in the terminal to view the help documentation